Software Requirements Specification (SRS)

Project: E-Commerce & Order Management System (Olist)
Course: CSN15101 · Software Engineering and Project Management
Team: Tarush Banke · Tanishk Varshney · Tanishq Mahajan
Group: TarushTanishkTanishq · **Instructor:** Komal Gupta
Version: 1.0 (mid-semester), structure based on IEEE 830

---

1. Introduction

1.1 Purpose

This document states what the E-Commerce & Order Management System must do. It is the basis for design (*Database_Design.pdf / Architecture_Diagram.pdf*), testing (*Test_Cases.pdf*) and planning (*Project_Plan.pdf*).

1.2 Scope

The system stores the order history of the Olist Brazilian marketplace (≈100k orders, 2016–2018) in a PostgreSQL database. Staff use a web application to manage orders, products, customers, sellers, reviews and categories, run reports, and see which orders are likely to be delivered late. The database itself enforces the business rules and access control.

1.3 Definitions

Term	Meaning
Order	A purchase by one customer, containing one or more order items
Order item	One product line in an order, fulfilled by one seller
RLS	Row-level security: PostgreSQL policies that filter rows per role
RBAC	Role-based access control
Late delivery	Delivered after the estimated delivery date promised at checkout
SQL panel	UI panel listing every SQL statement a user action executed

1.4 References

- Olist Brazilian E-Commerce Public Dataset (Kaggle, CC BY-NC-SA 4.0)
- IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications
- PostgreSQL 16 documentation

---

2. Overall description

2.1 Product perspective

A stand-alone, three-tier system: browser front end → Flask REST API → PostgreSQL 16. The ML model is trained offline by a Python script that reads features from and writes results to the database.

2.2 User classes

User class	Database role	Needs
Administrator	`olist_admin`	Full access, user management

User class	Database role	Needs
Operations manager	`olist_manager`	Manage orders, products, customers, sellers; run all reports
Data analyst	`olist_analyst`	Read-only reporting; no exact customer location
Seller	`olist_seller`	See and update only orders that contain their own items
Customer support	`olist_support`	Look up orders and customers (privacy view), answer reviews

2.3 Operating environment

Windows 10/11 (also macOS/Linux), PostgreSQL 16, Python 3.10+, any modern browser.

2.4 Design and implementation constraints

- Business rules must live in the DBMS (constraints, triggers, procedures), not only in application code.
- The UI must show the SQL it executes.
- Only free/open-source software; must run offline on a student laptop.
- The raw dataset may not be redistributed (not committed to Git).

2.5 Assumptions and dependencies

- The 9 Olist CSV files are available locally.
- PostgreSQL is installed with the `pgcrypto` extension (bundled with standard installers).

---

3. Functional requirements

ID	Requirement	Priority	Implemented in
FR-1	The system shall authenticate users by username and password; passwords are stored as bcrypt hashes and checked inside the database (`fn_login`).	High	`db/07` , `backend/app.py`
FR-2	After login, every request shall run under the PostgreSQL role mapped to the user's application role (`SET LOCAL ROLE`).	High	`backend/db.py`
FR-3	Users shall be able to create, search, view, update and delete **orders**, subject to their role.	High	Orders page
FR-4	Users shall be able to create, search, update and delete **products, customers, sellers, reviews and categories**.	High	CRUD pages
FR-5	Placing an order shall insert the order, its items and its payment **atomically** (`sp_place_order`); any failure rolls back everything.	High	`db/07`
FR-6	Order status shall only move forward (created → approved → invoiced → processing → shipped → delivered); canceled/unavailable are final.	High	`trg_validate_status_transition`
FR-7	Every order creation and status change shall be written to an audit log with the role that made it.	High	`trg_order_status_log`
FR-8	Delivered orders and orders with payments shall not be deletable.	Medium	`trg_block_order_delete`
FR-9	A review shall only be accepted for an order that has been delivered (or canceled/unavailable).	Medium	`trg_review_only_after_delivery`
FR-10	Placing an order shall reduce product stock; stock can never go negative.	Medium	`trg_reduce_stock` , CHECK

ID	Requirement	Priority	Implemented in
FR-11	Order search shall support filters (status, state, category, date range, rating, payment type) and show how many rows each option would return.	Medium	Orders page
FR-12	The system shall provide 16 predefined reports (joins, subqueries, aggregates, GROUP BY/HAVING, window functions, CTEs, set operations, full-text search), each with EXPLAIN ANALYZE and CSV export.	High	`backend/queries.py`
FR-13	Every page shall display the SQL statements executed, with row counts, time and errors.	High	all pages
FR-14	The UI shall show the ER model and the live schema (tables, keys, constraints, triggers, routines, indexes, policies, grants).	High	ER & schema page
FR-15	The system shall demonstrate atomicity, consistency, isolation levels and the lost-update problem (with and without `SELECT … FOR UPDATE`).	High	Transactions lab
FR-16	Sellers shall only see orders containing their own items; analysts shall not see customer coordinates.	High	RLS, column privileges
FR-17	The system shall predict, for an order, the probability of late delivery and the expected delivery time, and store predictions in the database.	High	`ml/`, `ml_prediction`
FR-18	A "predict only" action shall score a hypothetical order without saving it (transaction rolled back).	Medium	ML page
FR-19	Administrators shall be able to create application users with a role.	Low	`sp_create_user`
FR-20	A read-only SQL console shall let users run their own SELECT queries safely (read-only transaction, 15 s timeout).	Low	SQL console

---

4. Non-functional requirements

ID	Category	Requirement
NFR-1	Security	Least privilege: the app's login role `olist_app` owns no data privileges; all access goes through group roles.
NFR-2	Security	All user input is passed as bound parameters (no string-built SQL); passwords never leave the database.
NFR-3	Integrity	Data is stored in BCNF with PK, FK, UNIQUE and CHECK constraints on every table.
NFR-4	Reliability	Each web request is one transaction: commit on success, rollback on any error.
NFR-5	Performance	Order search and reports on the full dataset return in under 1 second on a typical laptop (B-tree, composite, partial and GIN indexes).
NFR-6	Usability	A new user can build and start the system with three scripts (`setup_db`, `train_model`, `run_app`).
NFR-7	Maintainability	SQL is split into one file per concern (`db/01` – `14`); the full database can be rebuilt from scratch in about a minute.
NFR-8	Portability	Runs on Windows, macOS and Linux; no build step for the front end.

ID	Category	Requirement
NFR-9	Transparency	The SQL shown in the UI is exactly the SQL the server executed (client-side parameter binding).
NFR-10	ML validity	The model is validated on time-ordered data (no future information in features) and reports ROC-AUC, PR-AUC and recall on held-out months.

---

5. Data requirements

- **Source:** 9 CSV files, ≈1.6 million rows in total (1.0 M of them geolocation points).
- **Cleaning:** geolocation reduced to one point per zip prefix with outliers removed; missing zip prefixes and category translations added; anomalies exposed in `v_data_quality`.
- **Retention:** the database can be dropped and rebuilt from the CSVs at any time; the CSVs are not stored in the repository.

---

6. Use cases

ID	Actor	Use case	Main flow
UC-1	Any user	Log in	Enter credentials → `fn_login` checks bcrypt hash → role stored in session
UC-2	Manager	Place order	Pick customer, products, payment → `CALL sp_place_order` → order, items, payment inserted in one transaction → risk score shown
UC-3	Seller	Update order status	Open own order → choose next status → trigger validates and logs the change
UC-4	Manager	Cancel order	`CALL sp_cancel_order` → status set to canceled, audit row written
UC-5	Analyst	Run report	Choose Q1–Q16 → view rows, SQL, EXPLAIN plan → export CSV
UC-6	Support	Answer review	Search review → mark answered
UC-7	Admin	Create user	Enter username, password, role → `CALL sp_create_user`
UC-8	Any user	View ER model / schema	Click "ER model & schema"
UC-9	Manager	Check delivery risk	Open ML page → see high-risk orders, confusion matrix, what-if prediction

---

7. Traceability to course requirements

Course requirement	Covered by
Entities, relationships, ER/EER → relational schema	Design §3, `db/01_schema.sql`
Keys, FDs, normalization to 3NF/BCNF	Design §4, `db/14_normalization_demo.sql`
DDL/DML, joins, nested/correlated queries, aggregates, GROUP BY/HAVING	FR-12, `db/09`, `db/12`
Views, indexes, triggers, procedures	FR-6 … FR-10, `db/04` – `07`
Transactions, ACID, concurrency	FR-5, FR-15, `db/10`, `db/11`
Security & RBAC	FR-1, FR-2, FR-16, `db/08`
Front end with insert/search/update/delete/reports and SQL shown	FR-3, FR-4, FR-11 … FR-14

Course requirement	Covered by
AI/ML integrated with DB and UI	FR-17, FR-18